



Master SESI

UM4IN108 - Systèmes Programmables (FPGA)

Centrale DCC sur FPGA

**Boudih Rabab 21318814
Ben Brahim Rym 21202841**

Encadrants : Denoulet Julien, Rhouni Amine

Date : 20 Mai 2026

Table des matières

1	Introduction	1
2	Protocole DCC	1
3	Architecture de la centrale DCC	1
3.1	Génération des bits DCC (bit 0 et bit 1)	1
3.2	Registre DCC	3
3.3	Machine à états globale	4
3.4	Système complet de la centrale DCC	5
3.5	Générateur de trames test	6
4	Création de l'IP centrale DCC	6
5	Création de l'IP Accéléromètre	7
6	Intégration des IPs au système Microblaze	8
7	Développement logiciel	8
7.1	Structure du projet logiciel	8
7.2	Déroulement du programme et Interface utilisateur	9
8	Bilan d'avancement	10
9	Conclusion	10

1 Introduction

Ce projet porte sur la conception d'une centrale de commande DCC sur carte FPGA Basys 3, capable de contrôler des locomotives à partir d'une interface utilisateur tout en respectant les contraintes du protocole DCC.

Notre travail s'est déroulé en deux grandes phases. La première consiste à développer les modules matériels en VHDL pour générer et envoyer les trames DCC. La seconde phase porte sur l'intégration de cette centrale dans un système Microblaze, où un programme en C permet à l'utilisateur de contrôler les trains via les boutons et les interrupteurs de la carte.

Ce rapport présente les différentes étapes de développement et les résultats obtenus, de la description matérielle jusqu'à la validation sur la plateforme trains.

2 Protocole DCC

Le protocole DCC est un standard utilisé dans le domaine ferroviaire pour transmettre des commandes numériques aux locomotives via les rails. Dans le cadre de notre projet, nous avons implémenté ce protocole sur le FPGA afin de contrôler individuellement plusieurs trains.

Les commandes sont transmises sous forme de trames composées de quatre champs : un préambule d'au moins 14 bits à 1, un champ d'adresse identifiant la locomotive, un champ de commande contenant l'instruction à exécuter (vitesse, direction, fonctions), et enfin un octet de contrôle calculé par XOR pour détecter les éventuelles erreurs de transmission.

Les bits sont codés par des impulsions de durées spécifiques : un bit à 1 est représenté par une demi-période à 0 de 58 μ s suivie d'une demi-période à 1 de 58 μ s, et un bit à 0 par une demi-période à 0 de 100 μ s suivie d'une demi-période à 1 de 100 μ s.

Le respect de ces durées est indispensable pour garantir un décodage correct par les locomotives. Les trames sont émises en continu, espacées d'un intervalle de 6 ms.

3 Architecture de la centrale DCC

Cette section présente l'implémentation de la centrale DCC sur le FPGA. L'objectif est de décrire les différents modules que nous avons développés en VHDL, leur fonctionnement individuel ainsi que leur interconnexion au sein de l'architecture globale du système.

3.1 Génération des bits DCC (bit 0 et bit 1)

Comme rappelé précédemment, chaque bit DCC est codé par des impulsions de durées spécifiques : 58 μ s pour un bit à 1 et 100 μ s pour un bit à 0.

Pour répondre à ces contraintes, nous avons créé deux modules différents : un module afin de générer les bits à 0 et un module générant les bits à 1. Ces deux modules reposent cependant sur la même logique. Nous allons donc présenter en détail le fonctionnement du module de génération du bit à 1, le module de génération du bit à 0 étant basé sur la même architecture, avec uniquement les paramètres temporels différents.

Description de la machine à états

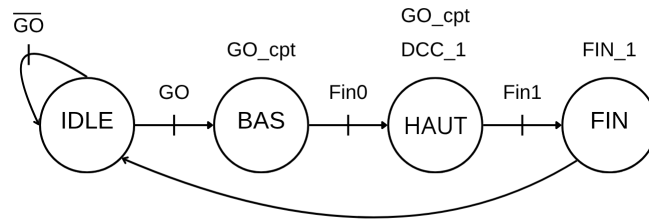


FIGURE 1 – Machine à état du bloc DCC_BIT1

Cette machine à états décrit le fonctionnement du module de génération du bit à 1 du protocole DCC. Le module est initialement dans l'état IDLE, où il attend un signal de démarrage GO.

Lorsque GO est activé, la machine passe dans l'état BAS, où le signal de sortie est maintenu à l'état bas pendant 58 μs via le compteur GO_cpt. À l'issue de cette durée, elle passe dans l'état HAUT, durant lequel le signal est maintenu à l'état haut pendant 58 μs , également contrôlée par le même compteur.

Le compteur utilisé est le composant `compteur_1`, cadencé par l'horloge `clk_1M`. Il est activé par GO_cpt et repose sur un comptage de 0 à 115. Il génère un signal Fin0 à la valeur 57 (fin de la première demi-période), puis un signal Fin1 à 115, entraînant la remise à zéro du compteur en fin de cycle. Le passage en état FIN marque la fin de génération du bit, la machine revient à l'état IDLE pour le bit suivant.

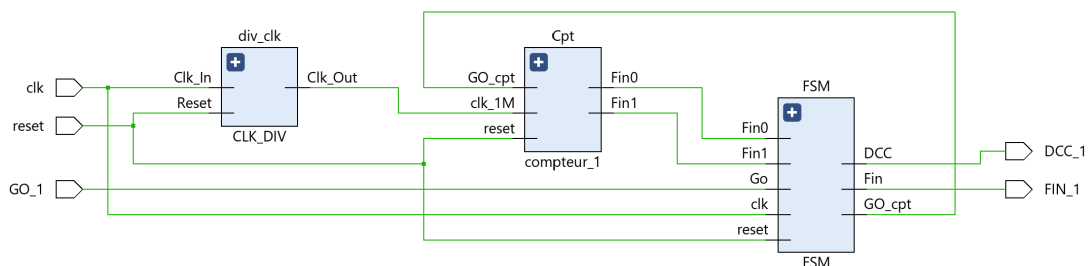


FIGURE 2 – Schéma RTL du bloc DCC_BIT1

Le schéma RTL permet de visualiser clairement la synchronisation entre la machine à états et le compteur, assurant la génération correcte d'un bit du protocole DCC. L'architecture du module repose sur trois blocs principaux : un diviseur d'horloge, le compteur `compteur_1` et une machine à états FSM. Le diviseur génère l'horloge lente `clk_1M` à partir de l'horloge système, permettant un comptage précis des durées imposées par le protocole DCC.

Simulation

Afin de valider le fonctionnement de ce module, nous avons réalisé une simulation comportementale. La Figure 3 présente une vue globale de la simulation où l'on peut observer que le module génère bien des bits à 1 de manière répétée, avec la machine à états qui alterne correctement entre les états S1 et S2 à chaque génération.

La Figure 4 présente un zoom sur la fin d'un cycle de génération. On peut y voir que lorsque le compteur atteint la valeur 115, le signal **Fin1** du compteur est bien levé, ce qui déclenche le passage en état **S3** puis la remise à zéro du compteur. Le signal **FIN_1** est ensuite généré pour indiquer à la **MAE** globale que la transmission du bit est terminée, avant de repasser en **S0** pour attendre le prochain **GO_1**.

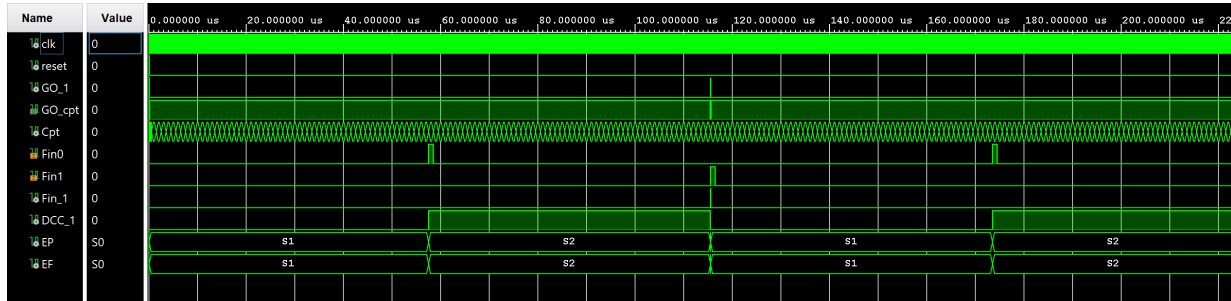


FIGURE 3 – Chronogramme de simulation du module DCC_BIT1

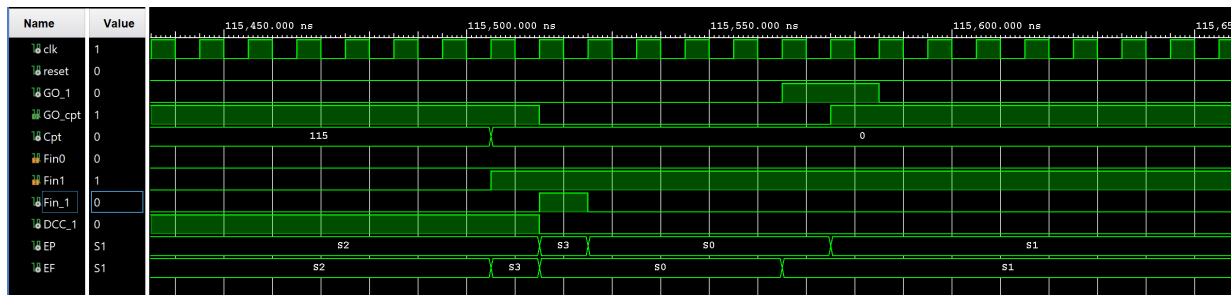


FIGURE 4 – Zoom sur la fin d'un cycle de génération du bit à 1

3.2 Registre DCC

Le registre DCC est un composant central de l'architecture. Son rôle est de stocker la trame DCC à transmettre, puis de la décaler bit par bit vers la gauche afin que la **MAE** puisse envoyer chaque bit séquentiellement vers les modules **DCC_BIT0** et **DCC_BIT1**.

Le registre est de taille 51 bits, ce qui correspond à la longueur maximale d'une trame DCC dans notre implémentation. Il est commandé par un signal **COM** sur 2 bits qui détermine son comportement à chaque front montant de l'horloge. Lorsque **COM** vaut "10", le registre charge la trame complète fournie par le Générateur de Trames. Lorsque **COM** vaut "11", il effectue un décalage vers la gauche d'un bit, en insérant un '0' en poids faible. Les valeurs "00" et "01" correspondent à un état inactif où le registre conserve son contenu.

Le bit de poids fort du registre correspond au premier bit à transmettre, ce qui est cohérent avec le protocole DCC qui impose une transmission du poids fort vers le poids faible.

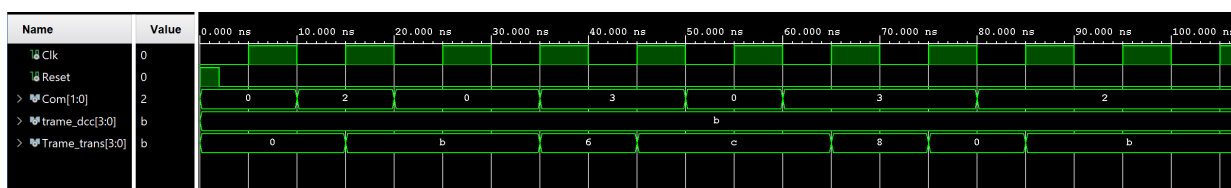


FIGURE 5 – Chronogramme de simulation du registre DCC

Cette Figure présente la simulation du registre DCC avec une entrée sur 4 bits. On peut observer que lorsque COM passe à "10", le registre charge bien la trame $b = "1011"$, et lorsque COM passe à "11", le décalage s'effectue correctement avec Trame_trans qui évolue bien de b vers $6 = "0110"$, $c = "1100"$ et $8 = "1000"$. On remarque également que les états inactifs $\text{COM}="00"$ et $\text{COM}="01"$ ne perturbent pas le contenu du registre, qui conserve bien sa valeur courante.

3.3 Machine à états globale

La machine à états globale est le module qui contrôle l'ensemble du système de la centrale DCC. C'est elle qui coordonne le fonctionnement de tous les autres blocs : elle charge la trame dans le registre DCC, détermine quel bit doit être envoyé, lance la génération de ce bit via les modules DCC_BIT0 ou DCC_BIT1, et gère la temporisation entre deux trames.

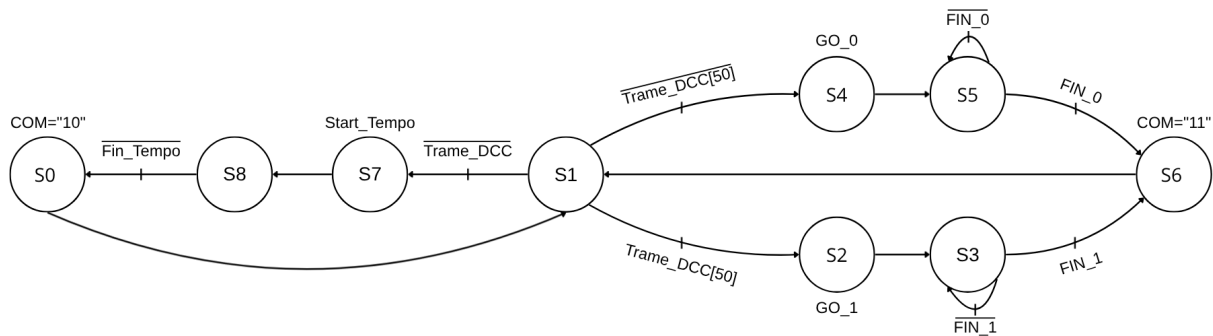


FIGURE 6 – Machine à état globale

La machine comporte 9 états dont le fonctionnement est le suivant :

S0 est l'état initial. La MAE y charge la trame dans le registre DCC en positionnant COM à "10", puis passe immédiatement à S1.

S1 est l'état de lecture du bit courant. La MAE examine le bit de poids fort du registre ($\text{Trame_DCC}[50]$) pour déterminer quel module appeler. Si ce bit vaut 1, elle passe à S2 pour générer un bit à 1. Si ce bit vaut 0, elle passe à S4 pour générer un bit à 0.

S2 et S3 gèrent la génération d'un bit à 1. En S2, la MAE active le module DCC_BIT1 via le signal GO_1. En S3, elle attend la fin de la génération en surveillant le signal FIN_1. Tant que FIN_1 n'est pas reçu, elle reste en S3. Une fois FIN_1 reçu, elle passe à S6.

S4 et S5 fonctionnent de manière symétrique pour la génération d'un bit à 0, avec les signaux GO_0 et FIN_0.

S6 est l'état de décalage. La MAE positionne COM à "11" pour décaler le registre d'un bit, puis retourne en S1 pour traiter le bit suivant.

S1 vérifie également si la trame est entièrement transmise. Si c'est le cas, elle passe à S7 au lieu de continuer la transmission.

S7 démarre la temporisation de 6 ms entre deux trames en activant le signal Start_Tempo, puis passe en S8, qui attend la fin de la temporisation en surveillant le signal Fin_Tempo. Une fois celui-ci reçu, la MAE retourne en S0 pour charger et transmettre la nouvelle trame.

Ce fonctionnement en boucle permet de générer en permanence la trame DCC courante sur la sortie SORTIE_DCC, conformément aux exigences du protocole qui imposent une émission continue espacée de 6 ms entre chaque trame.

3.4 Système complet de la centrale DCC

Après avoir validé individuellement chaque module, nous les avons assemblés et interconnectés pour former la centrale DCC complète. Le schéma RTL de la Figure 7 présente l'interconnexion de l'ensemble des modules. On peut y voir la MAE qui orchestre le fonctionnement global, le registre DCC qui stocke et décale la trame, les modules DCC_BIT0 et DCC_BIT1 qui génèrent les bits, le compteur de temporisation ainsi que le diviseur d'horloge. Les sorties des deux modules de génération de bits sont combinées via une porte OR pour produire le signal final SORTIE_DCC.

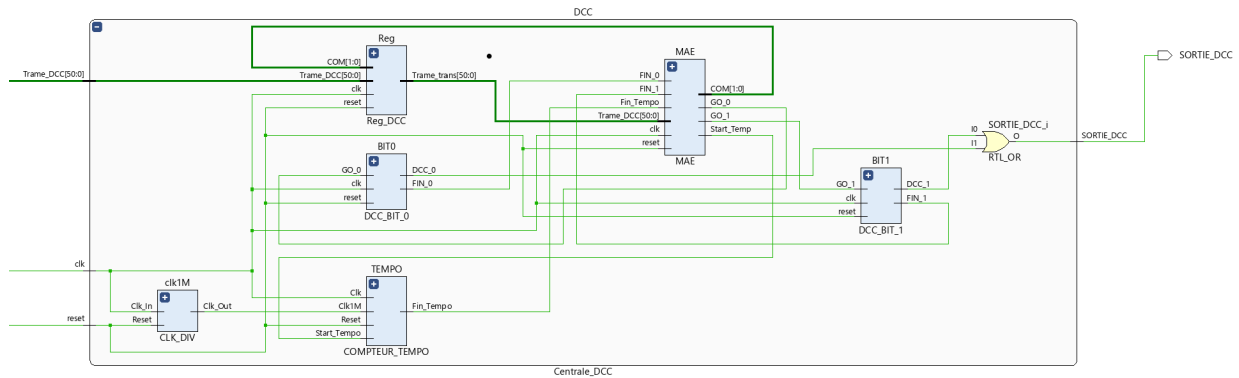


FIGURE 7 – Schéma RTL de la centrale DCC

Afin de mieux visualiser le comportement du système, nous avons effectué un premier test avec une trame réduite à 4 bits. On peut observer sur la Figure 8 que les contraintes temporelles sont bien respectées : un bit à 1 a bien une période de 116 μ s et un bit à 0 une période de 200 μ s. On constate également que le registre se décale correctement à chaque bit transmis, et qu'une fois la trame entièrement envoyée, la machine à états passe bien en S8 pour lancer la temporisation de 6 ms avant la prochaine trame.

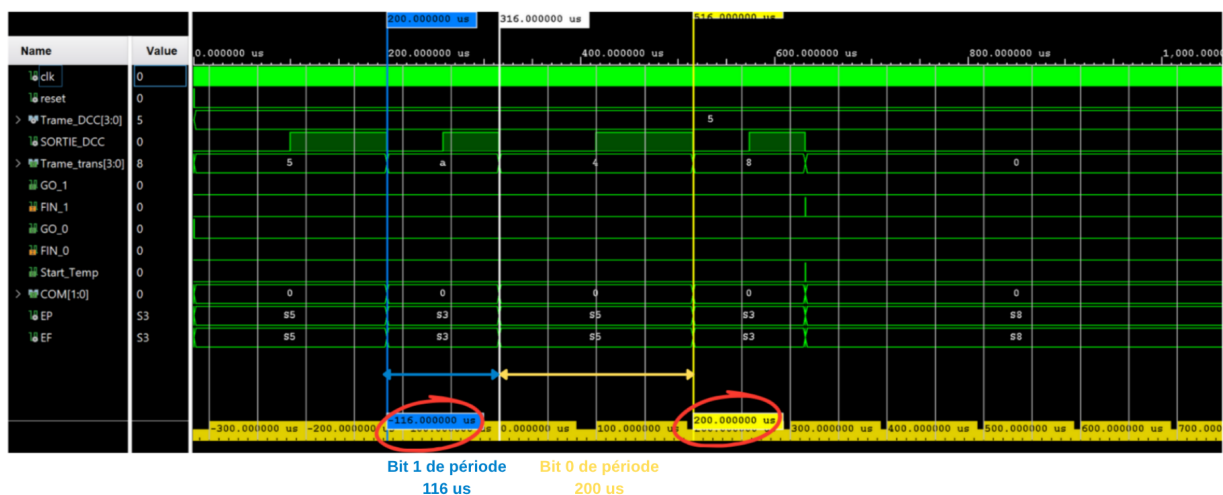


FIGURE 8 – Chronogramme de simulation de la centrale dcc avec une entrée sur 4bits

Les Figures 9 et 10 présentent deux zooms sur des instants clés de la simulation. On peut observer sur la Figure 9 que la MAE enchaîne correctement les états entre deux bits : elle passe bien par S6 pour décaler le registre avant de revenir en S1 pour traiter le bit suivant.

La Figure 10 confirme que la fin de trame est bien détectée, avec le passage en S7 qui lance la temporisation via `Start_Tempo`, puis en S8 où la machine attend la fin du délai de 6 ms avant de retransmettre la trame.

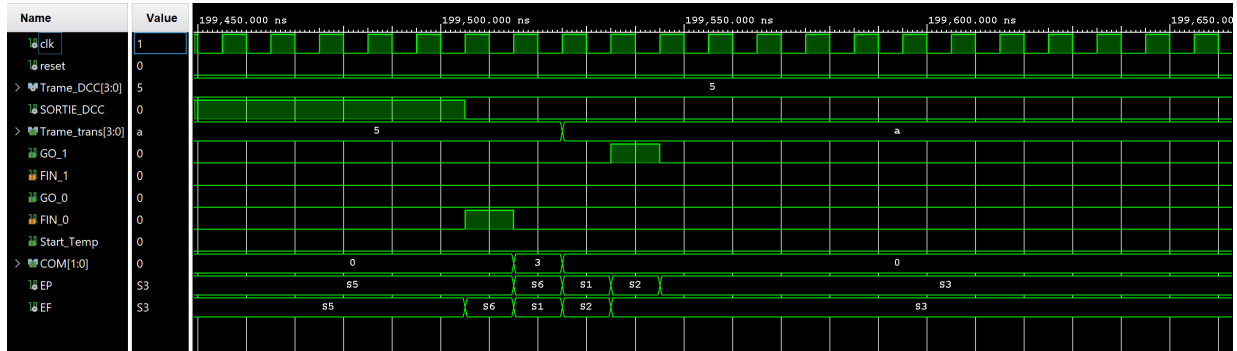


FIGURE 9 – Zoom sur la transition entre deux bits

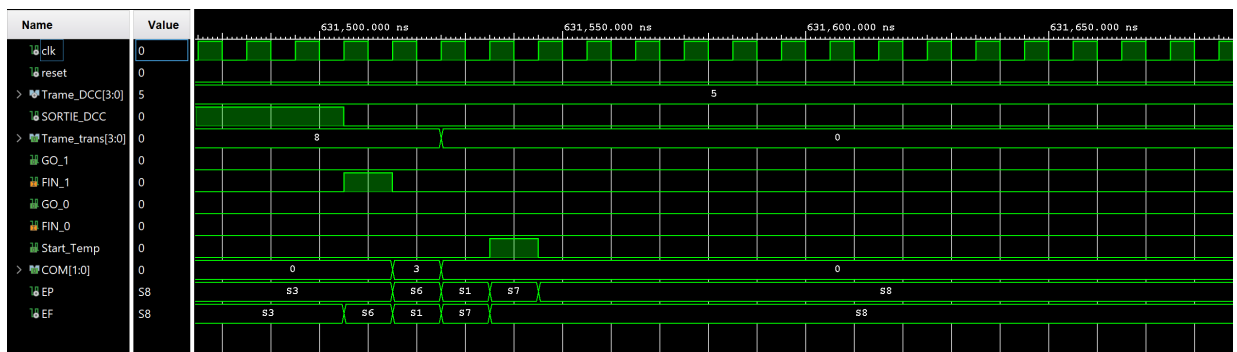


FIGURE 10 – Zoom sur la fin de trame et le lancement de la temporisation

3.5 Générateur de trames test

Le générateur de trames de test est un module temporaire dont le rôle est de valider le fonctionnement du hardware avant l'intégration avec le Microblaze. Il permet de générer des trames DCC prédéfinies en fonction de la configuration des interrupteurs de la carte.

Nous avons codé plusieurs trames correspondant à différentes commandes, comme par exemple le contrôle de la vitesse ou l'activation du klaxon... Selon la valeur lue sur les interrupteurs, le module sélectionne la trame correspondante et la transmet au registre DCC via le signal `Trame_DCC[50:0]`. Ce module nous a permis de tester et valider l'ensemble de la chaîne matérielle, depuis la génération de la trame jusqu'à l'émission du signal DCC en sortie, sans avoir besoin du Microblaze. Il sera remplacé dans la partie suivante par un AXI Wrapper permettant la communication avec le Microblaze via le bus AXI.

4 Création de l'IP centrale DCC

Pour intégrer la centrale DCC dans le système Microblaze, nous avons packagé notre architecture sous forme d'IP en utilisant l'outil de création d'IP de Vivado. L'IP comprend deux fichiers VHDL principaux. Le premier, `centrale_dcc_ip_v1_0`, est le fichier top de l'IP. Le second, `centrale_dcc_ip_v1_0_S00_AXI`, implémente le protocole AXI pour que l'IP puisse être reliée au bus, et instancie directement notre module Centrale DCC qui implémente la fonctionnalité de l'IP.

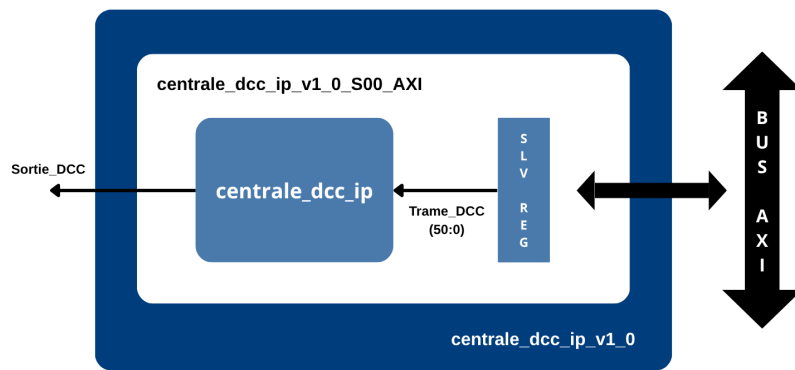


FIGURE 11 – Architecture interne de l'IP Centrale DCC

La trame DCC de 51 bits est répartie sur deux registres de 32 bits : les bits 50 à 19 sont stockés dans `slv_reg0`, et les bits 18 à 0 dans les bits de poids faible de `slv_reg1`.

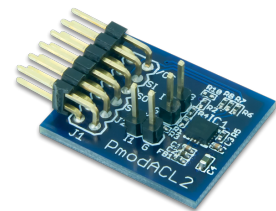
Nous avons également dû adapter le signal de reset. Le bus AXI fournit un reset actif au niveau bas (`S_AXI_ARESETN`), alors que les modules de notre centrale utilisent un reset actif au niveau haut. Nous avons donc inversé ce signal dans notre module.

5 Création de l'IP Accéléromètre

Pour cette partie, l'objectif était d'intégrer l'accéléromètre ADXL362 dans notre système Microblaze sous forme d'IP AXI. Deux fichiers VHDL nous ont été fournis : `SPI_If.vhd` pour la communication SPI et `ADXL362Ctrl.vhd` qui s'occupe de lire les données de l'accéléromètre et de les mettre à disposition sur 12 bits pour chaque axe (X, Y, Z) ainsi que la température.

La création de l'IP suit la même procédure que celle vue pour l'IP Centrale DCC. On a donc généré le wrapper AXI avec Vivado puis on a modifié les fichiers pour instancier `ADXL362Ctrl` et connecter ses sorties aux registres de l'IP.

- `slv_reg0` : axe X
- `slv_reg1` : axe Y
- `slv_reg2` : axe Z
- `slv_reg3` : température
- `slv_reg4` : signal `Data_Ready`



Concernant le reset, après vérification du code de `ADXL362Ctrl`, le port `RESET` est actif au niveau bas comme `S_AXI_ARESETN`, donc on les a connectés directement sans avoir besoin d'inverser le signal.

Un problème qu'on a rencontré est que les `slv_reg` ne peuvent pas être affectés depuis deux endroits différents en VHDL, ce qui provoque un conflit de driver. Pour régler ça, on a utilisé des signaux intermédiaires (`accel_x_reg`, `accel_y_reg`, etc.) qui récupèrent les données de l'accéléromètre et qui sont lus dans le process de lecture AXI à la place des `slv_reg`.

Enfin les ports SPI (`SCLK`, `MOSI`, `MISO`, `SS`) ont été déclarés comme ports externes dans le block design et ont été ajoutés dans le fichier XDC pour les relier au `PmodACL2` branché sur le connecteur JA de la carte Basys3.

6 Intégration des IPs au système Microblaze

Une fois les deux IPs développées packagées, nous les avons intégrées dans un système Microblaze via le block design de Vivado. La Figure 12 présente le block design complet du système. On peut y voir le processeur Microblaze connecté à l'IP Centrale DCC ainsi qu'à l'IP accéléromètre `my_acc`, toutes les deux reliées via l'interconnect AXI.

Pour gérer les entrées/sorties de la carte, nous avons ajouté deux blocs GPIO : un premier bloc `sw_boutton` qui regroupe les interrupteurs et les boutons, et un second bloc `sept_segment_led` qui gère l'afficheur 7 segments et les leds. Ces blocs permettent au Microblaze de lire l'état des entrées et de contrôler les sorties de la carte via le bus AXI.

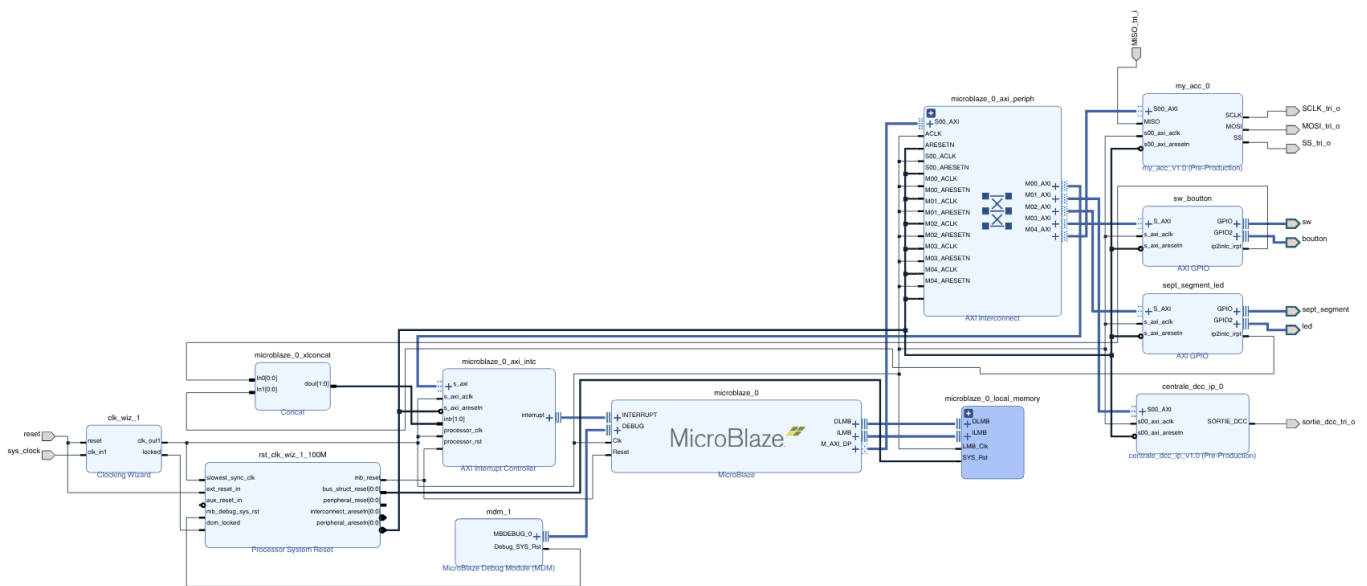


FIGURE 12 – Block design du système Microblaze

7 Développement logiciel

7.1 Structure du projet logiciel

Le code C est organisé en trois fichiers principaux. Le fichier `dcc.c` contient toutes les fonctions de construction et d'envoi des trames DCC : construction des trames de vitesse, de fonctions F0-F4, F5-F12 et F13-F20, ainsi que la fonction `envoyer_trame` qui écrit la trame dans les registres de l'IP via le bus AXI.

Le fichier `sept_segment.c` gère l'affichage sur les afficheurs 7 segments et fournit des fonctions d'affichage pour chaque mode (adresse, vitesse, fonctions).

Enfin le fichier `main.c` contient la boucle principale qui lit les entrées, gère la navigation entre les états et appelle les fonctions des deux fichiers précédents, et assure également la lecture des données de l'accéléromètre pour contrôler la vitesse et la direction du train en inclinant la carte.

7.2 Déroulement du programme et Interface utilisateur

Le programme principal est organisé autour d'une machine à états à 4 états. La dernière trame validée est transmise en permanence dans la boucle principale pour alimenter les trains en continu.

L'état 0 est l'état de démarrage. L'utilisateur choisit l'adresse du train via les interrupteurs [SW2:SW0], L'afficheur montre "Ad XX" en temps réel. L'adresse peut aller de 1 à 7, l'adresse 0 est un cas particulier qui permet de contrôler tous les trains en même temps. Un appui sur **btnR** permet de passer à l'état suivant.



FIGURE 13 – Affichage de l'adresse du train

L'état 1 permet de choisir entre le mode vitesse (SW0 = 1) ou le mode fonctions (SW1 = 1). L'afficheur affiche "StEP" pour le mode vitesse et "Ft-" pour le mode fonctions. **btnR** confirme et passe à l'état correspondant.

L'état 2 gère la commande de vitesse. Les boutons **btnU** et **btnD** permettent d'incrémenter ou décrémenter la vitesse, **SW0** fixe la direction, 1 pour avancer et 0 pour reculer. L'afficheur montre "SXX A" pour marche avant et "SXX r" pour marche arrière. Un appui sur **btnC** construit et envoie la trame de vitesse. **btnR** permet de revenir à l'état 0.



FIGURE 14 – Affichage de la vitesse en marche arrière



FIGURE 15 – Affichage de la vitesse en marche avant

Dans l'état 3, **btnU** et **btnD** permettent de naviguer entre les fonctions F0 à F20. L'afficheur indique "F-XX" où XX est le numéro de la fonction courante. **SW0** sert à activer ou désactiver la fonction : à 1 elle est activée, à 0 elle est désactivée. Un appui sur **btnC** construit et envoie la trame correspondante. **btnR** permet de revenir à l'état 0.



FIGURE 16 – Affichage d'une fonction

En parallèle de ces états, l'accéléromètre est lu en permanence dans la boucle principale. Les données des axes X et Y sont récupérées depuis les registres de l'IP `my_acc`. Les valeurs sont sur 12 bits signés, donc on vérifie le bit 11 pour savoir si la valeur est négative et on fait l'extension de signe si besoin.

Pour l'axe X, si la valeur dépasse +300 le train accélère de 2 pas, et si elle descend en dessous de -300 il ralentit de 2 pas. Pour l'axe Y, si la valeur dépasse +300 la direction passe en marche avant, et en dessous de -300 elle passe en marche arrière. À chaque changement, une nouvelle trame de vitesse est construite et envoyée au train. Un délai de 30ms est ajouté après chaque envoi pour éviter d'envoyer trop de trames d'un coup.

8 Bilan d'avancement

L'ensemble du projet a été réalisé et validé. La partie matérielle est complète, tous les modules VHDL ont été développés, simulés et synthétisés sur la carte. On a d'abord validé la centrale DCC avec le générateur de trames de test, qui nous a permis de vérifier le bon fonctionnement du système sur la plateforme trains avant même d'intégrer le Microblaze.

La partie logicielle est également fonctionnelle. Le code C permet de contrôler les trains via les interrupteurs, les boutons et l'accéléromètre. L'ensemble du système a été validé sur la plateforme trains, on arrive bien à contrôler la vitesse, la direction et les fonctions des locomotives.

9 Conclusion

Ce projet nous a permis de mettre en pratique l'ensemble des notions vues en cours sur les systèmes programmables. On est partis de zéro en concevant chaque module VHDL de la centrale DCC, en les simulant et en les validant un par un avant de les assembler en un système complet.

La deuxième phase du projet nous a appris à packager notre architecture sous forme d'IP et à l'intégrer dans un système Microblaze via le bus AXI. On a aussi appris à prendre en main une architecture existante qu'on ne connaissait pas, comme c'était le cas pour l'IP accéléromètre dont les fichiers VHDL nous ont été fournis, et à créer son IP pour l'intégrer dans notre système.

Enfin, le développement du code C nous a permis de voir comment on pilote plusieurs IPs et GPIOs dans un système Microblaze à l'aide des drivers générés par Vivado, ce qui nous a donné une vision complète du fonctionnement d'un système embarqué mixte matériel/logiciel.